CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 4

Name: Prionti Das | **UID: 24BAI71005**
Branch: CSE-AIML(CS221) | **Section/Group: 24AIT_NTPP-3(B)**
Semester: 5th | **Date of Performance: 30/07/26**
Subject: Full Stack - II | **Subject Code: 24CSP-337**

EXPERIMENT – 4.1

1. AIM:

To design and implement an interactive calendar interface for scheduling and managing posts.

2. OBJECTIVE:

1. To design and implement an interactive calendar interface for scheduling and managing posts.
2. To implement calendar-based scheduling interfaces
3. To map structured data to temporal layouts
4. To enable user interactions such as drag-and-drop

3. SOFTWARE REQUIREMENTS:

1. React.js
2. Calendar library
3. Date utility libraries
4. Code Editor

4. THEORY:

Calendars are essential UI components for representing time-based data. In applications such as social media schedulers, events (posts) must be mapped to specific dates and time slots. A calendar system involves:

- Temporal data modeling (date, time, duration)
- Event mapping (linking posts to time slots)
- User interaction handling (click, drag, resize)

Interactive calendars enhance usability by allowing users to:

- Visually organize content
- Modify schedules dynamically
- Manage large datasets efficiently

5. IMPLEMENTATION/PROCEDURE:

1. Design calendar UI layout (day/week/month view)
2. Map post data to calendar events
3. Render events dynamically
4. Implement click interactions (view/edit)
5. Add drag-and-drop scheduling *(optional/advanced)*
6. Sync calendar with application state

6. CODE :

CalendarView.css :

```
.calendar-container {
  background: white;
  border-radius: 15px;
  padding: 20px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
}

.calendar-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
  flex-wrap: wrap;
  gap: 15px;
}

.calendar-header h2 {
  color: #2d3436;
  font-size: 1.8rem;
  margin: 0;
}

.calendar-stats {
  display: flex;
  gap: 15px;
}

.stat-badge {
  background: #f0f0f0;
  padding: 8px 16px;
  border-radius: 20px;
  font-size: 0.9rem;
  color: #2d3436;
  font-weight: 500;
}

.fc {
  font-family: 'Segoe UI', sans-serif;
}

.fc .fc-toolbar-title {
  font-size: 1.5rem;
  font-weight: 600;
  color: #2d3436;
}

.fc .fc-button {
  background: #667eea;
  border: none;
  border-radius: 8px;
  padding: 8px 16px;
  font-weight: 500;
```

```
  text-transform: capitalize;
  transition: all 0.3s ease;
 }

 .fc .fc-button\:hover {
  background: #764ba2;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(118, 75, 162, 0.4);
 }

 .fc .fc-button-primary\:not(\:disabled).fc-button-active {
  background: #764ba2;
 }

 .fc .fc-daygrid-day {
  transition: *background* 0.3s ease;
 }

 .fc .fc-daygrid-day\:hover {
  background: #f8f9fa;
 }

 .fc .fc-daygrid-day-number {
  font-weight: 500;
  color: #2d3436;
 }

 .fc .fc-event {
  border-radius: 8px;
  padding: 4px 8px;
  font-size: 0.85rem;
  cursor: pointer;
  transition: all 0.3s ease;
  border: 2px solid rgba(255, 255, 255, 0.5);
  font-weight: 500;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
 }

 .fc .fc-event\:hover {
  transform: scale(1.03);
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
 }

 .fc .fc-daygrid-event {
  margin: 2px 4px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
 }

 .modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
```

```
  background: rgba(0, 0, 0, 0.6);
  backdrop-filter: blur(5px);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 1000;
  animation: fadeIn 0.3s ease;
}

@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

.modal-content {
  background: white;
  border-radius: 20px;
  padding: 30px;
  width: 90%;
  max-width: 500px;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
  animation: slideUp 0.3s ease;
}

@keyframes slideUp {
  from {
    transform: translateY(50px);
    opacity: 0;
  }
  to {
    transform: translateY(0);
    opacity: 1;
  }
}

.modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
  padding-bottom: 15px;
  border-bottom: 2px solid #f0f0f0;
}

.modal-header h3 {
  margin: 0;
  color: #2d3436;
  font-size: 1.5rem;
}

.close-btn {
  background: none;
```

```
  border: none;
  font-size: 1.8rem;
  cursor: pointer;
  color: #999;
  transition: color 0.3s ease;
 }

 .close-btn\:hover {
  color: #e74c3c;
 }

 .modal-body {
  margin-bottom: 20px;
 }

 .form-group {
  margin-bottom: 15px;
 }

 .form-group label {
  display: block;
  margin-bottom: 5px;
  color: #2d3436;
  font-weight: 500;
  font-size: 0.9rem;
 }

 .form-group input,
 .form-group textarea {
  width: 100%;
  padding: 10px 12px;
  border: 2px solid #e0e0e0;
  border-radius: 10px;
  font-size: 1rem;
  transition: border-color 0.3s ease;
  font-family: inherit;
 }
 .form-group input\:focus,
 .form-group textarea\:focus {
  outline: none;
  border-color: #667eea;
 }

 .form-group input\:disabled,
 .form-group textarea\:disabled {
  background: #f8f9fa;
  cursor: not-allowed;
 }

 .disabled-input {
  background: #f8f9fa !important;
 }

 .modal-footer {
  display: flex;
  gap: 10px;
```

```
  justify-content: flex-end;
  padding-top: 15px;
  border-top: 2px solid #f0f0f0;
 }

 .btn {
  padding: 10px 20px;
  border: none;
  border-radius: 10px;
  font-size: 0.95rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.3s ease;
 }

 .btn\:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
 }

 .btn-edit {
  background: #3498db;
  color: white;
 }

 .btn-delete {
  background: #e74c3c;
  color: white;
 }

 .btn-save {
  background: #2ecc71;
  color: white;
 }

 .btn-cancel {
  background: #95a5a6;
  color: white;
 }

 @media (max-width: 768px) {
  .calendar-header {
   flex-direction: column;
   align-items: flex-start;
  }

  .calendar-stats {
   flex-wrap: wrap;
  }

  .modal-content {
   padding: 20px;
  }
 }

 /* Notification styles */
```

```
.notification {
  position: fixed;
  top: 20px;
  right: 20px;
  background: #28a745;
  color: white;
  padding: 15px 25px;
  border-radius: 10px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
  z-index: 9999;
  animation: slideIn 0.5s ease;
}

@keyframes slideIn {
  from {
    transform: translateX(100%);
    opacity: 0;
  }

  to {
    transform: translateX(0);
    opacity: 1;
  }
}
```

CalendarView . js :

```
import React, { useState, useRef } from 'react';
import FullCalendar from '@fullcalendar/react';
import dayGridPlugin from '@fullcalendar/daygrid';
import interactionPlugin from '@fullcalendar/interaction';
import timeGridPlugin from '@fullcalendar/timegrid';
import './CalendarView.css';

function CalendarView({ events, onAddEvent, onUpdateEvent, onDeleteEvent }) {
  // 1. STATE MANAGEMENT - Track modal visibility and form data
  const [showModal, setShowModal] = useState(false);
  const [modalMode, setModalMode] = useState('view'); // 'view', 'edit', 'add'
  const [formData, setFormData] = useState({
    id: '',
    title: '',
    date: '',
    time: '',
    description: ''
  });

  const calendarRef = useRef(null);

  // 2. EVENT HANDLERS - Click on event to view details
  const handleEventClick = (info) => {
    const event = {
      id: info.event.id,
      title: info.event.title,
      date: info.event.startStr,
      time: info.event.extendedProps.time || '12:00 PM',
      description: info.event.extendedProps.description || 'No description'
    };
    setFormData(event);
```

```
  setModalMode('view');
  setShowModal(true);
 };

 // 3. DATE CLICK - Add new event on date click
 const handleDateClick = (info) => {
  const dateStr = info.dateStr;
  const newEvent = {
   id: Date.now().toString(),
   title: 'New Scheduled Post',
   date: dateStr,
   time: '12:00 PM',
   description: 'Enter description here'
  };
  setFormData(newEvent);
  setModalMode('add');
  setShowModal(true);
 };

 // 4. DRAG AND DROP - Reschedule event
 const handleEventDrop = (info) => {
  const updatedEvent = {
   id: info.event.id,
   title: info.event.title,
   date: info.event.startStr,
   time: info.event.extendedProps.time || '12:00 PM',
   description: info.event.extendedProps.description || 'No description'
  };
  onUpdateEvent(updatedEvent);

  // Show notification
  const notification = document.createElement('div');
  notification.className = 'notification';
  notification.textContent = ` ✅ ${info.event.title} moved to ${new Date(info.event.startStr).toDateString()}`;
  document.body.appendChild(notification);
  setTimeout(() => notification.remove(), 3000);
 };

 // 5. EVENT RESIZE - Change event duration
 const handleEventResize = (info) => {
  const updatedEvent = {
   id: info.event.id,
   title: info.event.title,
   date: info.event.startStr,
   time: info.event.extendedProps.time || '12:00 PM',
   description: info.event.extendedProps.description || 'No description'
  };
  onUpdateEvent(updatedEvent);
 };

 // 6. MODAL CONTROLS - Close modal
 const handleCloseModal = () => {
  setShowModal(false);
 };

 // 7. FORM INPUT - Handle form changes
```

```
const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
};

// 8. SAVE EVENT - Add or update event
const handleSaveEvent = () => {
  if (modalMode === 'add') {
    onAddEvent({
      ...formData,
      id: Date.now().toString()
    });
  } else if (modalMode === 'edit') {
    onUpdateEvent(formData);
  }
  handleCloseModal();
};

// 9. DELETE EVENT - Remove event
const handleDeleteEvent = () => {
  if (window.confirm('Are you sure you want to delete this event?')) {
    onDeleteEvent(formData.id);
    handleCloseModal();
  }
};

// 10. FORMAT EVENTS - Convert to FullCalendar format
const calendarEvents = events.map(event => ({
  id: event.id,
  title: event.title,
  start: event.date,
  extendedProps: {
    time: event.time,
    description: event.description
  },
  backgroundColor: getRandomColor(),
  borderColor: '#ffffff',
  textColor: '#ffffff'
}));

// 11. UTILITY - Generate random colors
function getRandomColor() {
  const colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7', '#DDA0DD', '#FF8A5C'];
  return colors[Math.floor(Math.random() * colors.length)];
}

// 12. RENDER - Display the calendar
return (
  <div className="calendar-container">
    {/* Header with statistics */}
    <div className="calendar-header">
      <h2>📊 Content Calendar</h2>
      <div className="calendar-stats">
```

```
          <span className="stat-badge">
            📝 Total Posts: {events.length}
          </span>
          <span className="stat-badge">
            🗓️ This Month: {events.filter(e =>
              new Date(e.date).getMonth() === new Date().getMonth()
            ).length}
          </span>
        </div>
      </div>

      {/* FullCalendar Component */}
      <div className="calendar-wrapper">
        <FullCalendar
          ref={calendarRef}
          plugins={[dayGridPlugin, interactionPlugin, timeGridPlugin]}
          initialView="dayGridMonth"
          headerToolbar={{
            left: 'prev,next today',
            center: 'title',
            right: 'dayGridMonth,timeGridWeek,timeGridDay'
          }}
          events={calendarEvents}
          editable={true}
          selectable={true}
          selectMirror={true}
          dayMaxEvents={true}
          weekends={true}
          eventClick={handleEventClick}
          dateClick={handleDateClick}
          eventDrop={handleEventDrop}
          eventResize={handleEventResize}
          eventTimeFormat={{
            hour: '2-digit',
            minute: '2-digit',
            meridiem: 'short'
          }}
          height="auto"
          locale="en"
          buttonText={{
            today: 'Today',
            month: 'Month',
            week: 'Week',
            day: 'Day'
          }}
        />
      </div>

      {/* Modal for Event Details */}
      {showModal && (
        <div className="modal-overlay" onClick={handleCloseModal}>
          <div className="modal-content" onClick={(e) => e.stopPropagation()}>
            <div className="modal-header">
              <h3>
                {modalMode === 'view' && '📋 Event Details'}
                {modalMode === 'edit' && '✏️ Edit Event'}
```

```
            {modalMode === 'add' && '➕ Add New Event'}
          </h3>
          <button className="close-btn" onClick={handleCloseModal}>✕</button>
        </div>

        <div className="modal-body">
          <div className="form-group">
            <label>Title:</label>
            <input
              type="text"
              name="title"
              value={formData.title}
              onChange={handleInputChange}
              disabled={modalMode === 'view'}
              className={modalMode === 'view' ? 'disabled-input' : ''}
            />
          </div>

          <div className="form-group">
            <label>Date:</label>
            <input
              type="date"
              name="date"
              value={formData.date}
              onChange={handleInputChange}
              disabled={modalMode === 'view'}
              className={modalMode === 'view' ? 'disabled-input' : ''}
            />
          </div>

          <div className="form-group">
            <label>Time:</label>
            <input
              type="time"
              name="time"
              value={formData.time}
              onChange={handleInputChange}
              disabled={modalMode === 'view'}
              className={modalMode === 'view' ? 'disabled-input' : ''}
            />
          </div>

          <div className="form-group">
            <label>Description:</label>
            <textarea
              name="description"
              value={formData.description}
              onChange={handleInputChange}
              disabled={modalMode === 'view'}
              className={modalMode === 'view' ? 'disabled-input' : ''}
              rows="3"
            />
          </div>
        </div>

        <div className="modal-footer">
```

```
            {modalMode === 'view' && (
              <>
                <button className="btn btn-edit" onClick={() => setModalMode('edit')}>
                  ✏️ Edit
                </button>
                <button className="btn btn-delete" onClick={handleDeleteEvent}>
                  🗑️ Delete
                </button>
              </>
            )}
            {(modalMode === 'edit' || modalMode === 'add') && (
              <>
                <button className="btn btn-save" onClick={handleSaveEvent}>
                  💾 Save
                </button>
                <button className="btn btn-cancel" onClick={handleCloseModal}>
                  ❌ Cancel
                </button>
              </>
            )}
          </div>
        </div>
      </div>
    )}
  </div>
 );
}

export default CalendarView;
```

App . js :

```
import React, { useState } from 'react';
import CalendarView from './components/CalendarView';
import './App.css';

function App() {
 // 1. STATE - Sample events data
 const [events, setEvents] = useState([
   {
    id: "1",
    title: "📱 Instagram Post - Product Launch",
    date: "2026-08-05",
    description: "New summer collection launch",
    time: "10:00 AM"
   },
   {
    id: "2",
    title: "💼 LinkedIn Post - Industry Insights",
    date: "2026-08-08",
    description: "Share market trends article",
    time: "2:30 PM"
   },
   {
    id: "3",
    title: "📘 Facebook Campaign - Brand Awareness",
    date: "2026-08-12",
    description: "Video ad campaign for new product",
```

```
    time: "9:00 AM"
  },
  {
    id: "4",
    title: "🐦 Twitter Post - Daily Update",
    date: "2026-08-15",
    description: "Company milestone announcement",
    time: "11:00 AM"
  },
  {
    id: "5",
    title: "📸 Instagram Story - Behind Scenes",
    date: "2026-08-18",
    description: "Team culture showcase",
    time: "4:00 PM"
  }
]);

// 2. CRUD OPERATIONS - Add new event
const addNewEvent = (newEvent) => {
  setEvents([...events, newEvent]);
};

// 3. CRUD OPERATIONS - Update existing event
const updateEvent = (updatedEvent) => {
  setEvents(events.map(event =>
    event.id === updatedEvent.id ? updatedEvent : event
  ));
};

// 4. CRUD OPERATIONS - Delete event
const deleteEvent = (eventId) => {
  setEvents(events.filter(event => event.id !== eventId));
};

// 5. RENDER - Main application layout
return (
  <div className="App">
    <div className="app-container">
      <header className="app-header">
        <h1>🗓️ Post Scheduling Calendar</h1>
        <p>Manage and schedule your social media posts efficiently</p>
      </header>

      <div className="calendar-wrapper">
        <CalendarView
          events={events}
          onAddEvent={addNewEvent}
          onUpdateEvent={updateEvent}
          onDeleteEvent={deleteEvent}
        />
      </div>

      <footer className="app-footer">
        <p>© 2026 Post Scheduler | Drag & Drop to reschedule</p>
      </footer>
```

```
      </div>
    </div>
  );
}

export default App;
```

App . css :

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  min-height: 100vh;
}

.App {
  max-width: 1400px;
  margin: 0 auto;
  padding: 20px;
}

.app-container {
  background: white;
  border-radius: 20px;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
  overflow: hidden;
}

.app-header {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 30px 40px;
  text-align: center;
}

.app-header h1 {
  font-size: 2.5rem;
  margin-bottom: 10px;
  font-weight: 700;
}

.app-header p {
  font-size: 1.1rem;
  opacity: 0.9;
}

.calendar-wrapper {
  padding: 20px;
  background: #f8f9fa;
}

.app-footer {
```

```
  text-align: center;
  padding: 20px;
  color: #6c757d;
  border-top: 1px solid #dee2e6;
}

@media (max-width: 768px) {
  .app-header h1 {
    font-size: 1.8rem;
  }

  .calendar-wrapper {
    padding: 10px;
  }
}
```

index . js :

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

index . css :

```
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

code {
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',
    monospace;
}
```

7. OUTPUT :

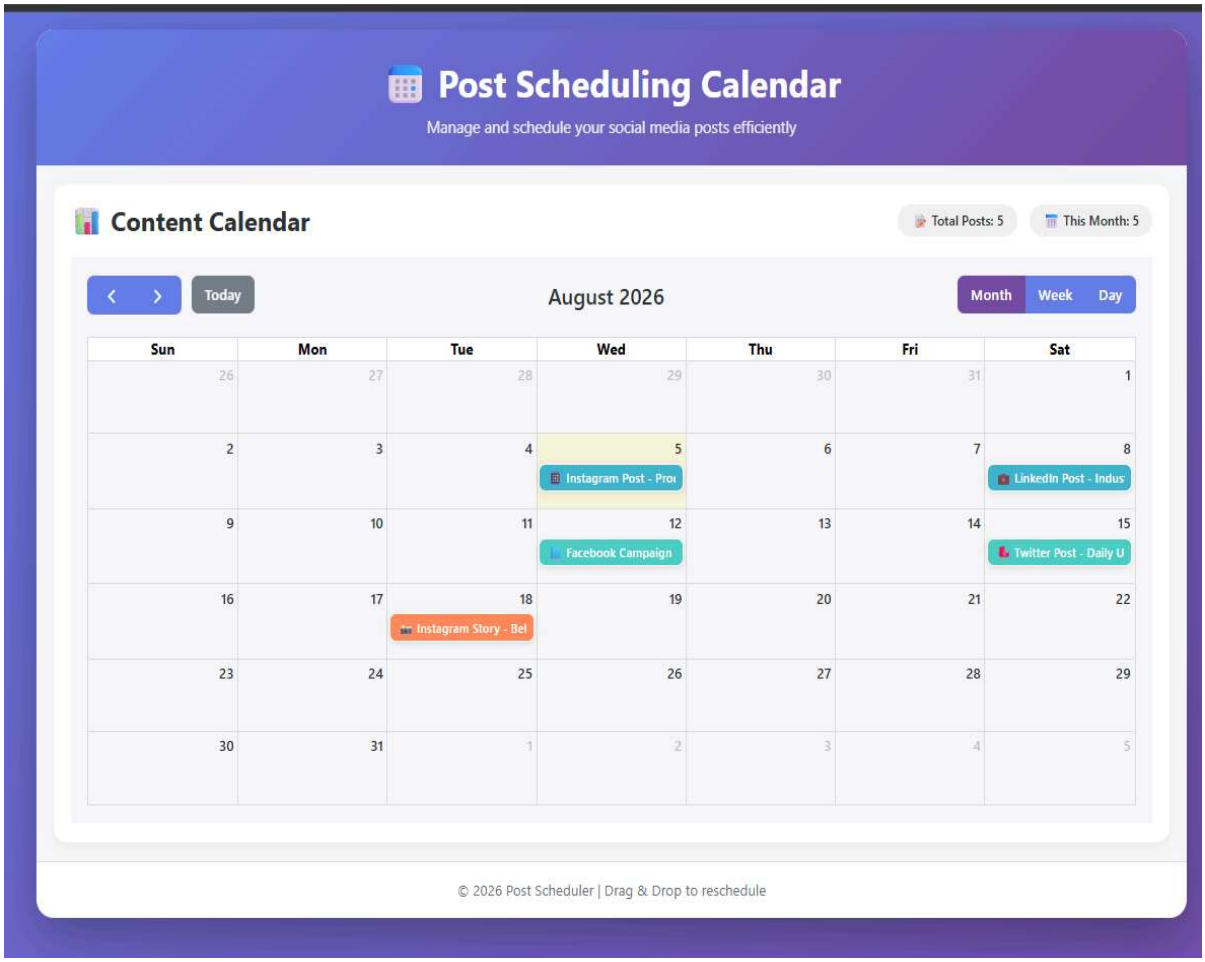


Fig 3.1: Post Scheduling Calendar Dashboard Output

The **Post Scheduling Calendar Dashboard** provides a visual interface for managing and scheduling social media posts. It displays all scheduled posts in a monthly calendar view, allowing users to organize content efficiently. The dashboard also shows the total number of scheduled posts, supports drag-and-drop rescheduling, and offers day, week, and month views for easy planning and management.

8. LEARNING OUTCOME:

- Design and implement an interactive calendar interface using React.
- Map structured post data to calendar events and time slots.
- Implement event handling for viewing and managing scheduled posts.
- Apply drag-and-drop functionality for dynamic event rescheduling.
- Synchronize calendar events with application state for efficient data management.
- Develop user-friendly scheduling systems for content planning and time-based applications.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 4.2

1. AIM:

To optimize rendering performance and implement testing strategies for interactive UI components

2. OBJECTIVE:

1. To understand performance bottlenecks in UI systems.
2. To optimize rendering using memoization techniques.
3. To reduce unnecessary re-renders in React applications.
4. To implement testing for UI components and application logic.

3. SOFTWARE REQUIREMENTS:

1. React.js
2. Jest
3. React Testing Library
4. Browser DevTools
5. Visual Studio Code
6. Node.js & npm

4. THEORY :

Interactive React applications such as calendars frequently update their UI based on user interactions. Without optimization, repeated rendering of components can negatively affect application performance and responsiveness.

Performance optimization techniques such as *React.memo, **useMemo, and **useCallback* help reduce unnecessary component re-renders and improve rendering efficiency. Memoization stores previously computed values and function references, ensuring computations occur only when required.

Testing is an essential part of frontend development. Unit testing verifies that UI components and application logic behave as expected under different scenarios, improving software reliability, maintainability, and stability.

By combining rendering optimization and testing, developers can build scalable, responsive, and reliable React applications

5. IMPLEMENTATION/PROCEDURE :

1. Create a React application and install the required dependencies.
2. Develop reusable React components for the calendar interface.
3. Identify components that re-render frequently.
4. Apply React.memo() to prevent unnecessary component re-rendering.
5. Use useMemo() to memoize expensive computations.
6. Use useCallback() to memoize event handler functions.
7. Optimize state updates to improve rendering performance.
8. Install Jest and React Testing Library.
9. Write unit test cases for calendar components.
10. Test user interactions such as event selection, drag-and-drop, and updates.
11. Execute test cases and verify successful execution.
12. Run the application and observe improved rendering performance

6. CODE :

App . js :

```
import React, { useState, useMemo, useCallback } from 'react';
import './App.css';
import Calendar from './components/Calendar';
import EventList from './components/EventList';

function App() {
 // Initial events data
 const [events, setEvents] = useState([
   { id: 1, title: 'Team Meeting', date: '2026-08-10', time: '10:00 AM' },
   { id: 2, title: 'Project Review', date: '2026-08-12', time: '2:00 PM' },
   { id: 3, title: 'Client Call', date: '2026-08-15', time: '11:30 AM' },
   { id: 4, title: 'Workshop', date: '2026-08-18', time: '9:00 AM' },
   { id: 5, title: 'Design Review', date: '2026-08-20', time: '3:00 PM' },
 ]);

 const [searchTerm, setSearchTerm] = useState('');
 const [selectedDate, setSelectedDate] = useState(null);

 // Memoize expensive computation - filtering events
 const filteredEvents = useMemo(() => {
   console.log('Filtering events...');
   return events.filter((event) => {
     const matchesSearch = event.title.toLowerCase().includes(searchTerm.toLowerCase());
     const matchesDate = selectedDate ? event.date === selectedDate : true;
     return matchesSearch && matchesDate;
   });
 }, [events, searchTerm, selectedDate]);

 // Memoize event handlers to prevent unnecessary re-renders
 const handleEventClick = useCallback((event) => {
   alert(`Event: ${event.title}\nDate: ${event.date}\nTime: ${event.time}`);
 }, []);

 const handleAddEvent = useCallback((newEvent) => {
   setEvents(prevEvents => [
     ...prevEvents,
     {
       id: prevEvents.length + 1,
       ...newEvent
     }
   ]);
 }, []);

 const handleDeleteEvent = useCallback((eventId) => {
   setEvents(prevEvents => prevEvents.filter(event => event.id !== eventId));
 }, []);

 const handleDateSelect = useCallback((date) => {
   setSelectedDate(date);
 }, []);

 return (
   <div className="app">
```

```
      <h1>📅 Calendar Application</h1>

      <div className="search-container">
        <input
          type="text"
          placeholder="Search events..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
          className="search-input"
        />
        {selectedDate && (
          <button
            onClick={() => setSelectedDate(null)}
            className="clear-filter-btn"
          >
            Clear Date Filter
          </button>
        )}
      </div>

      <div className="main-container">
        <Calendar
          events={events}
          onEventClick={handleEventClick}
          onDateSelect={handleDateSelect}
          selectedDate={selectedDate}
        />

        <EventList
          events={filteredEvents}
          onEventClick={handleEventClick}
          onDeleteEvent={handleDeleteEvent}
          onAddEvent={handleAddEvent}
        />
      </div>

      <div className="performance-info">
        <p>Total Events: {events.length} | Filtered Events: {filteredEvents.length}</p>
      </div>
    </div>
  );
}

export default App;
```

index . js :

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Calendar . js :

```
import React, { memo, useMemo } from 'react';
import './Calendar.css';

// React.memo prevents unnecessary re-renders
const Calendar = memo(function Calendar({
  events,
  onEventClick,
  onDateSelect,
  selectedDate
}) {
  console.log('Calendar component rendered');

  // Generate current month view
  const generateCalendarDays = useMemo(() => {
    const today = new Date();
    const year = today.getFullYear();
    const month = today.getMonth();

    const firstDay = new Date(year, month, 1);
    const lastDay = new Date(year, month + 1, 0);
    const daysInMonth = lastDay.getDate();
    const startingDayOfWeek = firstDay.getDay();

    const days = [];
    const weekdays = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

    // Add empty cells for days before first of month
    for (let i = 0; i < startingDayOfWeek; i++) {
      days.push(<div key={`empty-${i}`} className="calendar-cell empty"></div>);
    }

    // Add days of the month
    for (let day = 1; day <= daysInMonth; day++) {
      const dateStr = `${year}-${String(month + 1).padStart(2, '0')}-${String(day).padStart(2, '0')}`;
      const dayEvents = events.filter(event => event.date === dateStr);
      const isSelected = selectedDate === dateStr;
      const isToday = dateStr === `${year}-${String(month + 1).padStart(2, '0')}-${String(today.getDate()).padStart(2,
'0')}`;

      days.push(
        <div
          key={dateStr}
          className={`calendar-cell ${isSelected ? 'selected' : ''} ${isToday ? 'today' : ''}`}
          onClick={() => onDateSelect(dateStr)}
        >
          <div className="day-number">{day}</div>
          {dayEvents.length > 0 && (
            <div className="day-events">
              <span className="event-count">{dayEvents.length}</span>
              {dayEvents.map(event => (
                <div
                  key={event.id}
                  className="mini-event"
                  onClick={(e) => {
                    e.stopPropagation();
```

```
                onEventClick(event);
              }}
            >
              {event.title}
            </div>
          ))}
        </div>
      )}
    </div>
  );
 }

 return { days, weekdays };
}, [events, onEventClick, onDateSelect, selectedDate]);

return (
 <div className="calendar">
  <h2>🗓️ August 2026</h2>
  <div className="calendar-grid">
   <div className="calendar-header">
    {generateCalendarDays.weekdays.map(day => (
     <div key={day} className="weekday">{day}</div>
    ))}
   </div>
   <div className="calendar-body">
    {generateCalendarDays.days}
   </div>
  </div>
 </div>
);
});

export default Calendar;
```

EventList . js :

```
import React, { memo, useState, useCallback } from 'react';
import './EventList.css';

const EventList = memo(function EventList({
 events,
 onEventClick,
 onDeleteEvent,
 onAddEvent
}) {
 console.log('EventList component rendered');

 const [newEventTitle, setNewEventTitle] = useState('');
 const [newEventDate, setNewEventDate] = useState('');
 const [newEventTime, setNewEventTime] = useState('');

 const handleSubmit = useCallback((e) => {
  e.preventDefault();
  if (newEventTitle.trim() && newEventDate) {
   onAddEvent({
    title: newEventTitle.trim(),
    date: newEventDate,
    time: newEventTime || '12:00 PM'
```

```
    });
    setNewEventTitle('');
    setNewEventDate('');
    setNewEventTime('');
  }
}, [newEventTitle, newEventDate, newEventTime, onAddEvent]);

return (
  <div className="event-list">
    <h2>📋 Event List</h2>

    <form onSubmit={handleSubmit} className="add-event-form">
      <input
        type="text"
        placeholder="Event title..."
        value={newEventTitle}
        onChange={(e) => setNewEventTitle(e.target.value)}
        className="event-input"
      />
      <input
        type="date"
        value={newEventDate}
        onChange={(e) => setNewEventDate(e.target.value)}
        className="event-date-input"
      />
      <input
        type="time"
        value={newEventTime}
        onChange={(e) => setNewEventTime(e.target.value)}
        className="event-time-input"
      />
      <button type="submit" className="add-btn">Add Event</button>
    </form>

    <div className="events-container">
      {events.length === 0 ? (
        <p className="no-events">No events found</p>
      ) : (
        events.map(event => (
          <div key={event.id} className="event-item">
            <div
              className="event-content"
              onClick={() => onEventClick(event)}
            >
              <h3>{event.title}</h3>
              <p>🗓️ {event.date}</p>
              <p>🕐 {event.time}</p>
            </div>
            <button
              onClick={() => onDeleteEvent(event.id)}
              className="delete-btn"
            >
              Delete
            </button>
          </div>
        ))
```

```
        )}
      </div>
    </div>
  );
});

export default EventList;
```

Calendar . test . js :

```
import React from 'react';
import { render, screen, fireEvent } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import '@testing-library/jest-dom';
import Calendar from '../components/Calendar';
import EventList from '../components/EventList';
import App from '../App';

describe('Calendar Component Tests', () => {
  const mockEvents = [
    { id: 1, title: 'Test Meeting', date: '2026-08-10', time: '10:00 AM' },
    { id: 2, title: 'Project Review', date: '2026-08-12', time: '2:00 PM' },
  ];

  const mockHandlers = {
    onEventClick: jest.fn(),
    onDateSelect: jest.fn(),
  };

  beforeEach(() => {
    jest.clearAllMocks();
  });

  test('renders calendar component with correct heading', () => {
    render(
      <Calendar
        events={mockEvents}
        onEventClick={mockHandlers.onEventClick}
        onDateSelect={mockHandlers.onDateSelect}
        selectedDate={null}
      />
    );
    expect(screen.getByText(/🗓️ August 2026/i)).toBeInTheDocument();
  });

  test('displays weekday headers', () => {
    render(
      <Calendar
        events={mockEvents}
        onEventClick={mockHandlers.onEventClick}
        onDateSelect={mockHandlers.onDateSelect}
        selectedDate={null}
      />
    );
    expect(screen.getByText('Sun')).toBeInTheDocument();
    expect(screen.getByText('Mon')).toBeInTheDocument();
  });
```

```
  test('handles date selection when clicking on a calendar cell', async () => {
    render(
      <Calendar
        events={mockEvents}
        onEventClick={mockHandlers.onEventClick}
        onDateSelect={mockHandlers.onDateSelect}
        selectedDate={null}
      />
    );

    // Find the first calendar cell (day 1)
    const dayCells = screen.getAllByText('1');
    fireEvent.click(dayCells[0]);

    expect(mockHandlers.onDateSelect).toHaveBeenCalled();
  });
});

describe('EventList Component Tests', () => {
  const mockEvents = [
    { id: 1, title: 'Test Meeting', date: '2026-08-10', time: '10:00 AM' },
    { id: 2, title: 'Project Review', date: '2026-08-12', time: '2:00 PM' },
  ];

  const mockHandlers = {
    onEventClick: jest.fn(),
    onDeleteEvent: jest.fn(),
    onAddEvent: jest.fn(),
  };

  beforeEach(() => {
    jest.clearAllMocks();
  });

  test('renders event list with events', () => {
    render(
      <EventList
        events={mockEvents}
        onEventClick={mockHandlers.onEventClick}
        onDeleteEvent={mockHandlers.onDeleteEvent}
        onAddEvent={mockHandlers.onAddEvent}
      />
    );
    expect(screen.getByText('Test Meeting')).toBeInTheDocument();
    expect(screen.getByText('Project Review')).toBeInTheDocument();
  });

  test('shows no events message when list is empty', () => {
    render(
      <EventList
        events={[]}
        onEventClick={mockHandlers.onEventClick}
        onDeleteEvent={mockHandlers.onDeleteEvent}
        onAddEvent={mockHandlers.onAddEvent}
      />
    );
```

```
  expect(screen.getByText('No events found')).toBeInTheDocument();
});

test('handles event click', () => {
  render(
    <EventList
      events={mockEvents}
      onEventClick={mockHandlers.onEventClick}
      onDeleteEvent={mockHandlers.onDeleteEvent}
      onAddEvent={mockHandlers.onAddEvent}
    />
  );

  const eventElement = screen.getByText('Test Meeting');
  fireEvent.click(eventElement);

  expect(mockHandlers.onEventClick).toHaveBeenCalledWith(mockEvents[0]);
});

test('handles event deletion', () => {
  render(
    <EventList
      events={mockEvents}
      onEventClick={mockHandlers.onEventClick}
      onDeleteEvent={mockHandlers.onDeleteEvent}
      onAddEvent={mockHandlers.onAddEvent}
    />
  );

  const deleteButtons = screen.getAllByText('Delete');
  fireEvent.click(deleteButtons[0]);

  expect(mockHandlers.onDeleteEvent).toHaveBeenCalledWith(1);
});

test('handles adding new event', async () => {
  const user = userEvent.setup();
  render(
    <EventList
      events={mockEvents}
      onEventClick={mockHandlers.onEventClick}
      onDeleteEvent={mockHandlers.onDeleteEvent}
      onAddEvent={mockHandlers.onAddEvent}
    />
  );

  const titleInput = screen.getByPlaceholderText('Event title...');
  const dateInput = screen.getByPlaceholderText('YYYY-MM-DD');
  const addButton = screen.getByText('Add Event');

  await user.type(titleInput, 'New Test Event');
  await user.type(dateInput, '2026-08-25');

  fireEvent.change(dateInput, { target: { value: '2026-08-25' } });
  await user.click(addButton);
```

```
    expect(mockHandlers.onAddEvent).toHaveBeenCalledWith({
      title: 'New Test Event',
      date: '2026-08-25',
      time: '12:00 PM'
    });
  });
});

describe('App Integration Tests', () => {
  test('renders calendar application with all components', () => {
    render(<App />);
    expect(screen.getByText('🗓️ Calendar Application')).toBeInTheDocument();
    expect(screen.getByText('🗓️ August 2026')).toBeInTheDocument();
    expect(screen.getByText('📋 Event List')).toBeInTheDocument();
  });

  test('filters events based on search input', async () => {
    const user = userEvent.setup();
    render(<App />);

    const searchInput = screen.getByPlaceholderText('Search events...');
    await user.type(searchInput, 'Team Meeting');

    await waitFor(() => {
      expect(screen.getByText('Team Meeting')).toBeInTheDocument();
    });
  });
});
```

package . json :

```
{
  "name": "calendar-experiment-final",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@testing-library/jest-dom": "^5.17.0",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": [
      "react-app",
      "react-app/jest"
    ]
  },
  "browserslist": {
    "production": [
```

```
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
 }
}
```

7. OUTPUT:

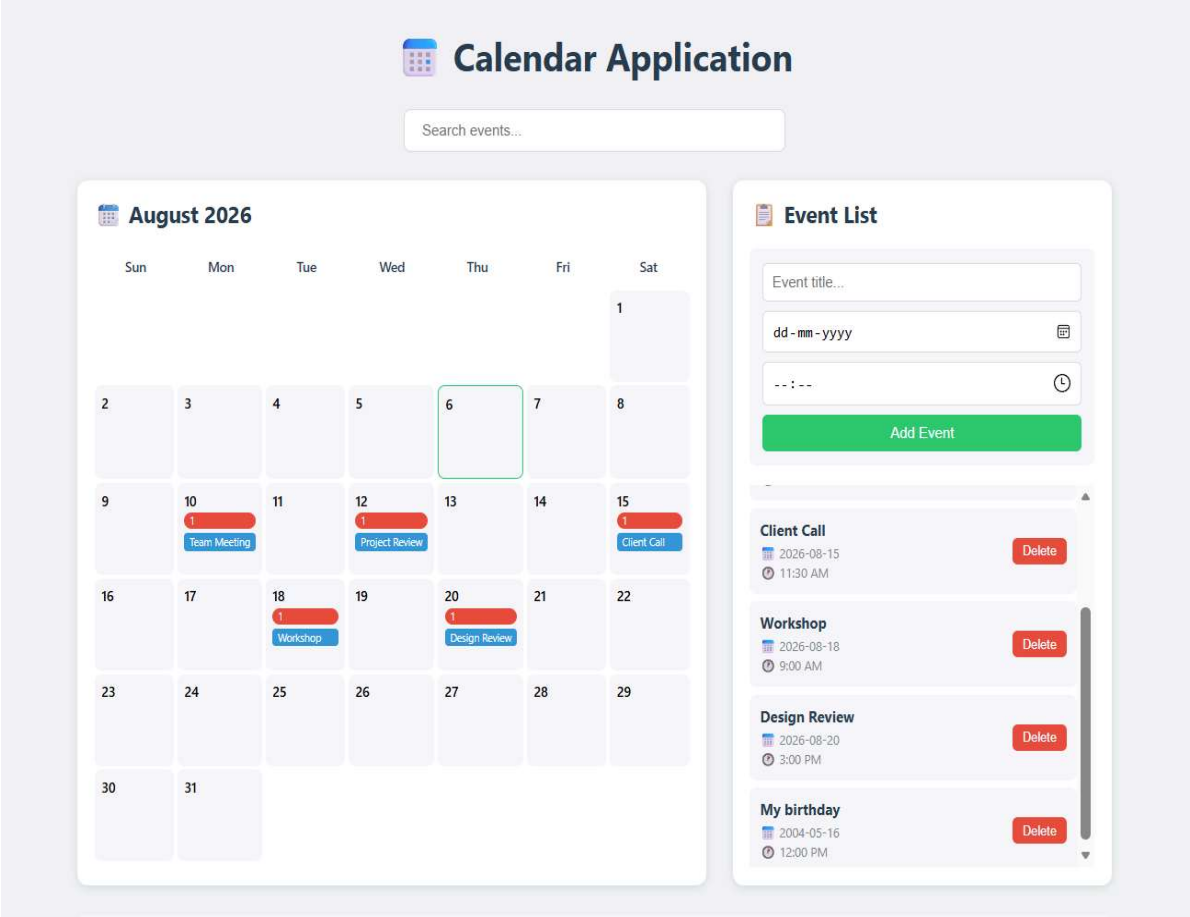


Fig 4.2:Calendar Application with Event Management System

The **Calendar Application** provides an interactive interface for creating, viewing, and managing events. Users can add events by entering the title, date, and time, search for specific events, and delete events when needed. The monthly calendar visually displays scheduled events, while the event list allows users to organize and manage their schedules efficiently.

8. LEARNING OUTCOME:

- Implement memoization using React.memo, useMemo and useCallback.
- Reduce unnecessary component re-renders to improve application efficiency.
- Optimize state updates for interactive UI components.
- Develop and execute unit tests using Jest and React Testing Library.
- Validate component functionality and user interactions through testing.
- Build scalable, optimized, and maintainable React applications following performance engineering and software testing principles.